

Part 1: The Phantom Bug

- Write a program that tosses 50,000,000 random (x, y) points into a 1×1 square to approximate π ($\pi \approx 4 \times \frac{\text{hits}}{\text{total}}$).
- Split the loop across 4 native Java threads.
- Have all 4 threads update a shared variable: `static long totalHits = 0;` via `totalHits++`.
- **Deliverable:** Run it 5 times. Record the value of π .
- *The Lesson:* The code runs fast, but gives $\pi \approx 1.8$ to 2.4 because threads overwrite each other's memory writes (data race).

```
PS C:\Users\orynb\AppData\Local\Program
Run 1: pi = 1.70732 (hits: 21341501)
Run 2: pi = 1.75497 (hits: 21937164)
Run 3: pi = 1.71623 (hits: 21452883)
Run 4: pi = 1.87093 (hits: 23386634)
Run 5: pi = 1.71778 (hits: 21472274)
```

Part 2: The Synchronization Trap

- Fix the race condition by adding `synchronized` to the increment or using `AtomicLong.incrementAndGet()`.
- **Deliverable:** Measure execution time vs. a plain, single-threaded for loop.
- *The Lesson:* The answer is now accurate (≈ 3.1415), but it runs drastically slower than a single thread because cores spend 95% of their cycles stalling on memory bus locks.

```
PS C:\Users\orynb\AppData\Local\Programs\Microsoft VS Code> & C:\Use
Single Process:           Time = 6713 ms, pi = 3.14154
Synchronized Multi-process: Time = 209857 ms, pi = 3.14119
PS C:\Users\orynb\AppData\Local\Programs\Microsoft VS Code> █
```

Part 3: OpenMP-Style Reduction

- Eliminate shared-state locks. Give each thread a **private, local counter** (conceptually identical to `#pragma omp parallel for reduction(+:totalHits)`).
- Accumulate local hits inside the thread, then add the partial sums together *only once* at the very end when joining threads.
- **Deliverable:** Benchmark total execution time for $T \in \{1, 2, 4, 8, 16, 32\}$ threads on their machine with 100,000,000 iterations.

- Fill out this grid:

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread (TN T1)	Efficiency (T/Speedup)
1	13156	1.0x	100%
2	6647	1.98x	99%
4	4332	3.04x	75.9%
8	2314	5.68x	71.1%
16	2055	6.04x	40.0%
32	2351	5.60x	17.5%

Questions

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

My CPU only has 8 physical cores. Going from 8 to 16 threads relies on logical cores (Hyper-Threading/SMT), which don't actually double the hardware's computing power—they just let each core switch between two execution pipelines when one is idle. Since this Monte Carlo calculation is purely CPU-bound and constantly uses the math blocks (ALU), there weren't many idle cycles for Hyper-Threading to take advantage of. Looking at my results, execution time barely dropped from 2314 ms to 2055 ms, and the extra thread management and context switching tanked the overall efficiency down to 40%.

2. Why was the synchronized version in Part 2 slower than running on one single core?

The synchronized approach creates massive thread contention because all 4 threads are constantly fighting over a single lock every time they record a hit. Instead of working in parallel, they end up waiting in line to update the exact same memory location. On top of that, forcing multiple cores to synchronize a single shared variable triggers constant cache line bouncing (the CPU has to keep invalidating L1/L2 caches across cores to maintain memory coherence). All that locking overhead and memory bus stalling wastes way more time than just letting a single core run through the loop sequentially without any locks.

Upload your solutions before the class end-time to the Git repo that you created for benchmarks. Share repo access with @sufyanism (GitHub/GitLab).